EXP-1 SETTING UP THE PYTHON ENVIRONMENT AND LIBRARIES

AIM

- Create a new notebook for Python.
- Write and execute Python code.
- Create new cells for Code and Markdown.
- Demonstrate the application of Jupyter Widgets.

PROGRAM

```
import ipywidgets as widgets
from IPython.display import display

slider = widgets.IntSlider(
    value=50,
    min=0,
    max=100,
    step=1,
    description='Sara'
)

output = widgets.Output()

def on_value_change(change):
    output.clear_output()
    with output:
        print(f'Slider value is {change["new"]}')
```

```
slider.observe(on_value_change, names='value')


display(slider, output)
```

OUTPUT



RESULT

Thus, the given program was successfully executed and the Jupyter Widget slider was created and demonstrated successfully.

EXP:2 EDA – DATA IMPORT AND EXPORT

AIM

To import data from CSV files, Excel files, SQL databases, and web sources, handle different data formats, and export a DataFrame to an Excel file.

PROGRAM

```
import pandas as pd

import requests

from io import StringIO

csv_data =
"""Loan_ID,Gender,Married,Dependents,Education,Self_Employed,ApplicantIncome,Coap
plicantIncome,LoanAmount,Loan_Amount_Term,Credit_History,Property_Area

LP001002,Male,Yes,0,Graduate,No,5720,0,110,360,1,Urban

LP001003,Male,Yes,1,Graduate,No,3076,1500,126,360,1,Urban

LP001005,Male,Yes,0,Not Graduate,No,5000,2000,208,360,1,Urban

LP001006,Male,Yes,0,Graduate,No,2340,2546,100,360,1,Urban

LP001008,Male,No,0,Graduate,No,3276,0,78,360,1,Urban"""


df_temp = pd.read_csv(StringIO(csv_data))

df_temp.to_csv("/content/test_Y3wMUE5_7gLdaTN.csv", index=False)


excel_data = {

  "Year": [1992, 1993, 1994, 1995, 1996],

  "Stocks": [0.4381, 6.0800, 6.2532, 4.2102, 4.4384],

  "T-Bills": [0.0066, 0.0096, 0.0100, 0.0085, 0.0091],

  "T-Bonds": [0.0354, 0.0504, 0.0450, 0.0320, 0.0435]

}
```

```
df_temp2 = pd.DataFrame(excel_data)
df_temp2.to_excel("/content/Historicalinvesttemp.xlsx", index=False)

df_csv = pd.read_csv("/content/test_Y3wMUE5_7gLdaTN.csv")
print(df_csv.head())

df_excel = pd.read_excel(
   "/content/Historicalinvesttemp.xlsx",
   sheet_name="Sheet1"
)
print(df_excel.head())

url = "https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)"

headers = {
   "User-Agent": "Mozilla/5.0"
}

response = requests.get(url, headers=headers)

if response.status_code == 200:
   tables = pd.read_html(StringIO(response.text))
   df_web = tables[0]
   print(df_web.head())
else:
```

```
df_web = pd.DataFrame({
    "Country": ["United States", "China", "Germany", "Japan", "India"],
    "GDP": [29184, 18743, 4659, 4186, 3942]
})
print(df_web.head())
df = pd.read_excel("/content/Historicalinvesttemp.xlsx")
df.to_excel("/content/weeeeee.xlsx", index=False)
print(df.head())
```

OUTPUT

```
    Loan_ID Gender Married  Dependents     Education Self_Employed  \
0  LP001002   Male     Yes           0      Graduate            No
1  LP001003   Male     Yes           1      Graduate            No
2  LP001005   Male     Yes           0  Not Graduate            No
3  LP001006   Male     Yes           0      Graduate            No
4  LP001008   Male      No           0      Graduate            No

   ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0             5720                  0         110               360
1             3076               1500         126               360
2             5000               2000         208               360
3             2340               2546         100               360
4             3276                  0          78               360

   Credit_History Property_Area
0               1         Urban
1               1         Urban
2               1         Urban
3               1         Urban
4               1         Urban
   Year  Stocks  T-Bills  T-Bonds
0  1992  0.4381   0.0066   0.0354
1  1993  6.0800   0.0096   0.0504
2  1994  6.2532   0.0100   0.0450
3  1995  4.2102   0.0085   0.0320
4  1996  4.4384   0.0091   0.0435
                                                   0
0  Largest economies in the world by nominal GDP ...
   Year  Stocks  T-Bills  T-Bonds
0  1992  0.4381   0.0066   0.0354
1  1993  6.0800   0.0096   0.0504
2  1994  6.2532   0.0100   0.0450
3  1995  4.2102   0.0085   0.0320
4  1996  4.4384   0.0091   0.0435
```

RESULT

Thus, the given program was successfully executed. Data was imported from CSV, Excel, and web sources, and the DataFrame was successfully exported to an Excel file.

EXP-3 EDA – DATA CLEANING

AIM

To handle missing values, remove duplicate records, convert data types, maintain data consistency, and normalize numerical data.

PROGRAM

```
import pandas as pd

import numpy as np

from sklearn.preprocessing import MinMaxScaler, StandardScaler


data = {

    'Gender': ['Male', 'Female', 'Male', None, 'Female', 'Male', 'Male'],

    'Married': ['Yes', 'No', 'Yes', 'Yes', None, 'No', 'Yes'],

    'Dependents': ['0', '1', '3+', None, '2', '0', '0'],

    'Education': ['Graduate', 'Graduate', 'Not Graduate', 'Graduate', 'Graduate', 'Not Graduate',
'Graduate'],

    'Self_Employed': ['No', 'Yes', None, 'No', 'No', 'Yes', 'No'],

    'ApplicantIncome': [5720, 3076, 5000, 2340, 3276, 4500, 5720],

    'CoapplicantIncome': [0, 1500, 2000, 2546, 0, 1000, 0],

    'LoanAmount': [110, 126, None, 100, 78, 120, 110],

    'Credit_History': [1, 1, None, 0, 1, 1, 1],

    'Property_Area': ['Urban', 'Urban', 'Urban', 'Urban', 'Urban', 'Rural', 'Urban']

}


df = pd.DataFrame(data)


print("Initial Data")
```

```
print(df)

print("\nMissing Values")
print(df.isnull().sum())

for col in ['Gender', 'Married', 'Dependents', 'Self_Employed']:
    df[col] = df[col].fillna(df[col].mode()[0])

df['LoanAmount'] = df['LoanAmount'].fillna(df['LoanAmount'].median())

df['Credit_History'] = df['Credit_History'].fillna(
    df['Credit_History'].mode()[0]
)

df.drop_duplicates(inplace=True)

df['Dependents'] = df['Dependents'].replace('3+', 3)
df['Dependents'] = df['Dependents'].astype(int)

for col in ['Gender', 'Married', 'Education', 'Self_Employed', 'Property_Area']:
    df[col] = df[col].str.strip().str.capitalize()

min_max_scaler = MinMaxScaler()

scale_cols = [
    'ApplicantIncome',
    'CoapplicantIncome',
```

```
    'LoanAmount'
]

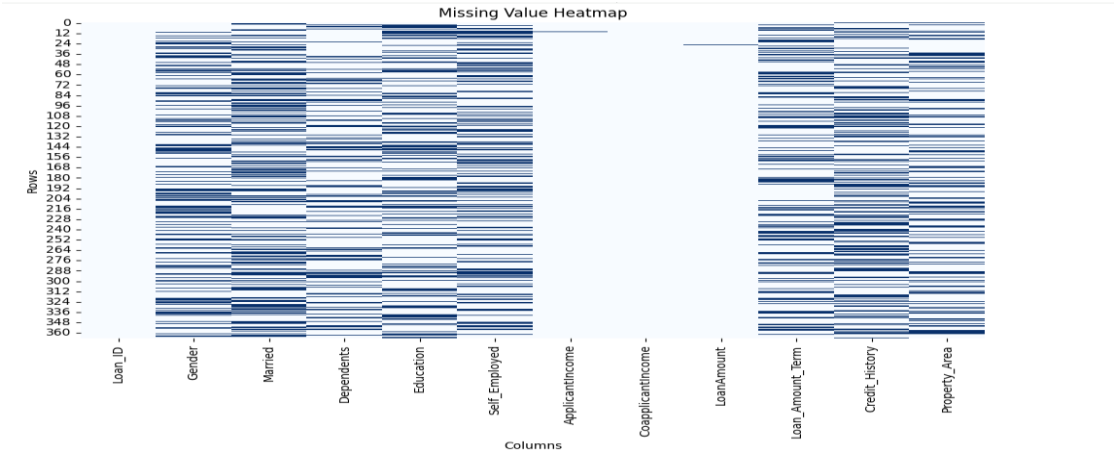
df[scale_cols] = min_max_scaler.fit_transform(df[scale_cols])

standard_scaler = StandardScaler()
df[['Credit_History']] = standard_scaler.fit_transform(
    df[['Credit_History']]
)
print("\nCleaned Data")
print(df)
print("\nFinal Data Information")
print(df.info())
```

OUTPUT

```
Initial Data Overview:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Loan_ID            367 non-null    object
 1   Gender             249 non-null    object
 2   Married            225 non-null    object
 3   Dependents         280 non-null    object
 4   Education          248 non-null    object
 5   Self_Employed      244 non-null    object
 6   ApplicantIncome    366 non-null    float64
 7   CoapplicantIncome  367 non-null    float64
 8   LoanAmount         366 non-null    float64
 9   Loan_Amount_Term   255 non-null    object
 10  Credit_History     234 non-null    object
 11  Property_Area      260 non-null    object
dtypes: float64(3), object(9)
memory usage: 34.5+ KB
None

Missing Values in Each Column:
Loan_ID               0
Gender              118
Married             142
Dependents           87
Education           119
Self_Employed       123
ApplicantIncome       1
CoapplicantIncome     0
LoanAmount            1
Loan_Amount_Term    112
Credit_History      133
Property_Area       107
dtype: int64
```




```
Removed 0 duplicate rows.

Credit_History:
   Credit_History
0       -0.641513
1        1.558816
2        1.558816
3       -0.641513
4        1.558816

Cleaned Data:
    Loan_ID  Gender Married  Dependents     Education Self_Employed  \
0  LP100000  Female      No           2      Graduate            No
1  LP100001  Female     Yes           0  Not graduate            No
2  LP100002    Male      No           1          None            No
3  LP100003    Male     Yes           3          None            No
4  LP100004  Female     Yes           1          None            No

   ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0         0.795461           0.461925    0.449799               120
1         0.004022           0.950490    0.550201               180
2         0.706263           0.949987    0.811245               180
3         0.412812           0.819050    0.248996               360
4         0.738437           0.462679    0.550201               360

   Credit_History Property_Area
0       -0.641513          None
1        1.558816         Urban
2        1.558816          None
3       -0.641513         Rural
4        1.558816         Rural

Final Data Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 12 columns):
```

```
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Loan_ID            367 non-null    object
 1   Gender             367 non-null    object
 2   Married            367 non-null    object
 3   Dependents         367 non-null    int64
 4   Education          248 non-null    object
 5   Self_Employed      367 non-null    object
 6   ApplicantIncome    366 non-null    float64
 7   CoapplicantIncome  367 non-null    float64
 8   LoanAmount         367 non-null    float64
 9   Loan_Amount_Term   367 non-null    int64
 10  Credit_History     367 non-null    float64
 11  Property_Area      260 non-null    object
dtypes: float64(4), int64(2), object(6)
memory usage: 34.5+ KB
None
```

RESULT

Thus, the program was successfully executed. Missing values were handled, duplicate records were removed, data types were converted, categorical values were standardized, and numerical data was successfully normalized.

EXP-4 **EDA – DATA INSPECTION AND ANALYSIS**

AIM

To inspect and analyze the dataset using DataFrame operations and calculate descriptive statistics such as mean, median, mode, range, variance, and standard deviation.

PROGRAM

```
import pandas as pd


data = {
    'Gender': ['Male', 'Female', 'Male', 'Female', 'Male'],
    'Married': ['Yes', 'No', 'Yes', 'Yes', 'No'],
    'Dependents': ['0', '1', '3+', '2', '0'],
    'Education': ['Graduate', 'Graduate', 'Not Graduate', 'Graduate', 'Graduate'],
    'Self_Employed': ['No', 'Yes', 'No', 'No', 'Yes'],
    'ApplicantIncome': [5720, 3076, 5000, 2340, 3276],
    'CoapplicantIncome': [0, 1500, 2000, 2546, 0],
    'LoanAmount': [110, 126, 208, 100, 78],
    'Property_Area': ['Urban', 'Urban', 'Urban', 'Rural', 'Urban']
}


df = pd.DataFrame(data)


print("First 5 Rows")
print(df.head())


print("\nLast 5 Rows")
print(df.tail())
```

```
print("\nData Information")
df.info()

print("\nDescriptive Statistics")
print(df.describe())

print("\nColumn Names")
print(df.columns)

categorical_cols = [
    'Gender',
    'Married',
    'Dependents',
    'Education',
    'Self_Employed',
    'Property_Area'
]

for col in categorical_cols:
    print(f"\nUnique values in {col}")
    print(df[col].unique())

df['Dependents'] = df['Dependents'].replace('3+', 3)

df['Dependents'] = pd.to_numeric(
    df['Dependents'],
```

```
    errors='coerce'
)
print("\nValue Counts of Dependents")
print(df['Dependents'].value_counts())
cols = [
    'ApplicantIncome',
    'CoapplicantIncome',
    'LoanAmount'
]
for col in cols:
    print(f"\nStatistics for {col}")
    print("Mean:", round(df[col].mean(), 2))
    print("Median:", round(df[col].median(), 2))
    print("Mode:", df[col].mode().values)
    print("Range:", round(df[col].max() - df[col].min(), 2))
    print("Variance:", round(df[col].var(), 2))
    print("Standard Deviation:", round(df[col].std(), 2))
```

OUTPUT

```
First 5 Rows
   Gender Married Dependents     Education Self_Employed  ApplicantIncome  \
0    Male     Yes          0      Graduate            No             5720
1  Female      No          1      Graduate           Yes             3076
2    Male     Yes         3+  Not Graduate            No             5000
3  Female     Yes          2      Graduate            No             2340
4    Male      No          0      Graduate           Yes             3276

   CoapplicantIncome  LoanAmount Property_Area
0                  0         110         Urban
1               1500         126         Urban
2               2000         208         Urban
3               2546         100         Rural
4                  0          78         Urban

Last 5 Rows
   Gender Married Dependents     Education Self_Employed  ApplicantIncome  \
0    Male     Yes          0      Graduate            No             5720
1  Female      No          1      Graduate           Yes             3076
2    Male     Yes         3+  Not Graduate            No             5000
3  Female     Yes          2      Graduate            No             2340
4    Male      No          0      Graduate           Yes             3276

   CoapplicantIncome  LoanAmount Property_Area
0                  0         110         Urban
1               1500         126         Urban
2               2000         208         Urban
3               2546         100         Rural
4                  0          78         Urban

Data Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
```

```
Data columns (total 9 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Gender             5 non-null      object
 1   Married            5 non-null      object
 2   Dependents         5 non-null      object
 3   Education          5 non-null      object
 4   Self_Employed      5 non-null      object
 5   ApplicantIncome    5 non-null      int64
 6   CoapplicantIncome  5 non-null      int64
 7   LoanAmount         5 non-null      int64
 8   Property_Area      5 non-null      object
dtypes: int64(3), object(6)
memory usage: 492.0+ bytes

Descriptive Statistics
       ApplicantIncome  CoapplicantIncome  LoanAmount
count         5.000000           5.000000    5.000000
mean       3882.400000        1209.200000  124.400000
std        1416.227665        1164.183491   49.867825
min        2340.000000           0.000000   78.000000
25%        3076.000000           0.000000  100.000000
50%        3276.000000        1500.000000  110.000000
75%        5000.000000        2000.000000  126.000000
max        5720.000000        2546.000000  208.000000

Column Names
Index(['Gender', 'Married', 'Dependents', 'Education', 'Self_Employed',
       'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount', 'Property_Area'],
      dtype='object')

Unique values in Gender
['Male' 'Female']

Unique values in Married
['Yes' 'No']

Unique values in Dependents
['0' '1' '3+' '2']

Unique values in Education
```

```
Column Names
Index(['Gender', 'Married', 'Dependents', 'Education', 'Self_Employed',
       'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount', 'Property_Area'],
      dtype='object')

Unique values in Gender
['Male' 'Female']

Unique values in Married
['Yes' 'No']

Unique values in Dependents
['0' '1' '3+' '2']

Unique values in Education
['Graduate' 'Not Graduate']

Unique values in Self_Employed
['No' 'Yes']

Unique values in Property_Area
['Urban' 'Rural']

Value Counts of Dependents
Dependents
0    2
1    1
3    1
2    1
Name: count, dtype: int64

Statistics for ApplicantIncome
Mean: 3882.4
Median: 3276.0
Mode: [2340 3076 3276 5000 5720]
Range: 3380
Variance: 2005700.8
Standard Deviation: 1416.23
```

```
Value Counts of Dependents
Dependents
0    2
1    1
3    1
2    1
Name: count, dtype: int64

Statistics for ApplicantIncome
Mean: 3882.4
Median: 3276.0
Mode: [2340 3076 3276 5000 5720]
Range: 3380
Variance: 2005700.8
Standard Deviation: 1416.23

Statistics for CoapplicantIncome
Mean: 1209.2
Median: 1500.0
Mode: [0]
Range: 2546
Variance: 1355323.2
Standard Deviation: 1164.18

Statistics for LoanAmount
Mean: 124.4
Median: 110.0
Mode: [ 78 100 110 126 208]
Range: 130
Variance: 2486.8
Standard Deviation: 49.87
```

RESULT

Thus, the given program was successfully executed. The dataset was inspected using DataFrame functions, unique values and value counts were identified, and measures of central tendency and dispersion were calculated successfully.

EXP -5 EDA – DATA VISUALIZATION

AIM

To explore and understand the underlying patterns, distributions, trends, and relationships in the data using different visualization techniques such as line charts, bar charts, and histograms.

PROGRAM

```
import matplotlib.pyplot as plt

import pandas as pd

import numpy as np


data = {

   'Category': ['A', 'B', 'C', 'D'],

   'Values': [3, 7, 5, 4]

}


df = pd.DataFrame(data)


print(df)


x = np.linspace(0, 10, 100)

y = np.sin(x)


plt.plot(x, y, label='Sine Wave')

plt.xlabel('X-axis')

plt.ylabel('Y-axis')

plt.title('Line Plot')

plt.legend()
```

```
plt.show()

categories = ['A', 'B', 'C', 'D']
values = [3, 7, 5, 4]

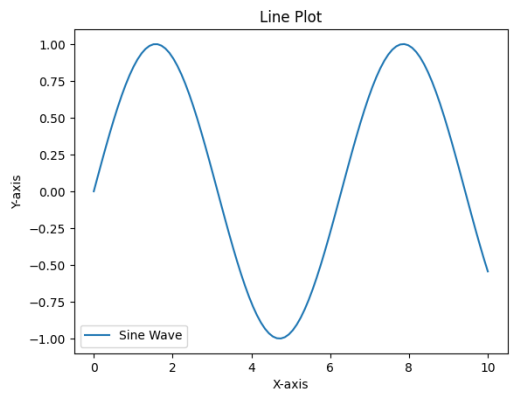
plt.bar(categories, values)
plt.xlabel('Category')
plt.ylabel('Values')
plt.title('Bar Plot')
plt.show()

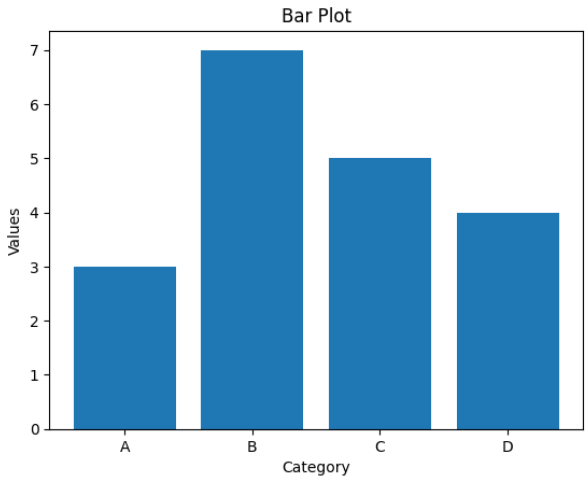
data = np.random.randn(1000)

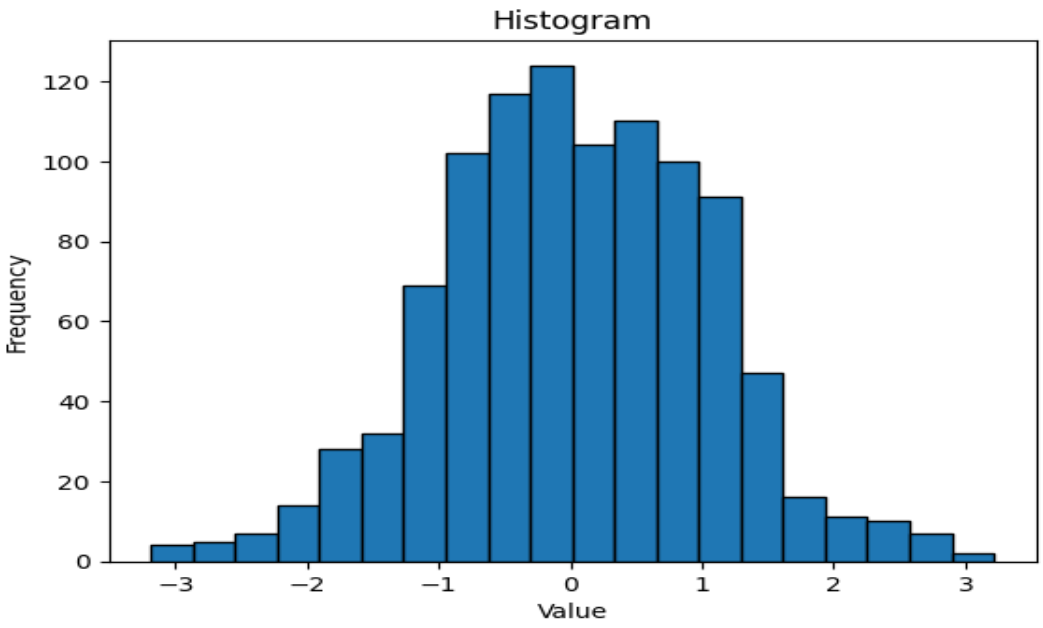
plt.hist(data, bins=20, edgecolor='black')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.title('Histogram')
plt.show()
```

OUTPUT

```
  Category  Values
0        A       3
1        B       7
2        C       5
3        D       4
```







RESULT

Thus, the given program was successfully executed. Line chart, bar chart, and histogram were generated successfully to visualize trends, category-wise comparisons, and frequency distributions in the data.

EXP:6 **DATA VISUALIZATION USING POWER BI**

AIM:

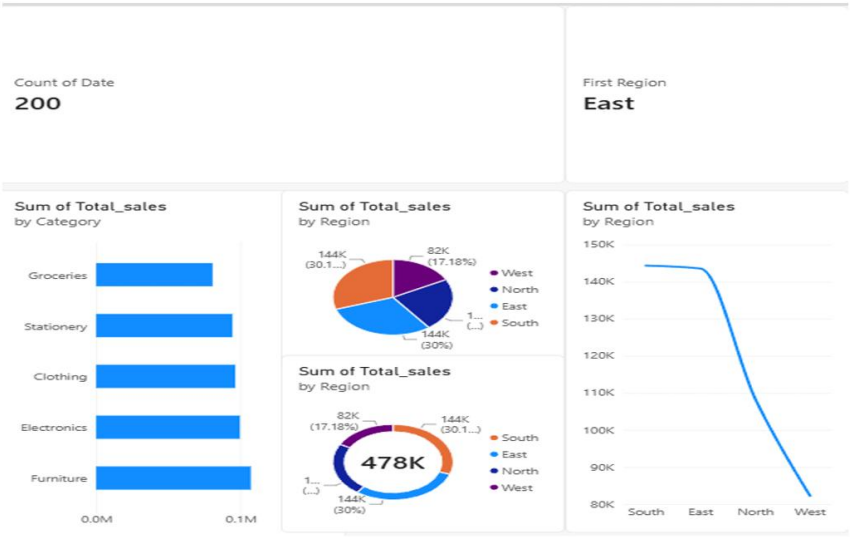
To learn the Power BI Desktop interface and workflow by connecting Excel/CSV data sources, performing basic data transformation, creating calculated columns and DAX measures, and developing interactive visualizations and dashboards.

PROCEDURE:

1. Open **Power BI Desktop** and explore the interface, including the Fields, Visualizations, Filters, Data, Report, and Model views.
2. Connect to the required data sources:
    - Select **Home → Get Data → Excel** and import SalesData.xlsx.
    - Select **Home → Get Data → Text/CSV** and import Targets.csv.
    - Select the required tables or sheets and click **Load**.
3. Open **Transform Data** to launch Power Query Editor.
4. Clean and transform the data:
    - Set the **Date** column to the Date data type.
    - Create a **Total Sales** calculated column using:
      Quantity × UnitPrice
    - Trim unnecessary spaces from text columns.
    - Remove duplicate records if required.
    - Ensure the Month field in Targets.csv has a consistent format.
5. Click **Close & Apply** to load the transformed data into Power BI.
6. Create basic visualizations:
    - **Bar Chart:** Display Sales by Category.
    - **Line Chart:** Display Monthly Sales Trend.
    - **Pie/Donut Chart:** Display Sales proportion by Region.
    - **Card Visuals:** Display Total Sales, Total Quantity, and Average Order Value.
    - **Table/Matrix:** Display the Top 10 Products based on Sales.

7. Add a **Region or Date slicer** to make the report interactive.
8. Arrange the visualizations neatly on a report page to create a dashboard-like layout.
9. Add suitable titles, data labels, tooltips, number formats, and other formatting options.
10. Save the Power BI report as a .pbix file.

OUTPUT:



RESULT:

Thus, the Sales and Target data were successfully connected, transformed, visualized, and presented as an interactive dashboard using Power BI.

EXP:7 DATA VISUALIZATION USING TABLEAU

AIM:

To understand the Tableau interface and perform data visualization by connecting Excel/CSV data sources, creating basic charts, applying filters and calculated fields, and developing an interactive dashboard and story.

PROCEDURE:

1. Open **Tableau Desktop/Tableau Public** and explore the Tableau interface, including the Start Page, Data Source, Worksheet, Dashboard, Columns, Rows, Marks, Filters, and Pages areas.
2. Connect to the required data source:
    - Select **Connect → Microsoft Excel** and import SalesData.xlsx.
    - Alternatively, select **Text File** and import Targets.csv.
3. In the **Data Source** page, select the required table or sheet and verify the data fields.
4. Create basic visualizations:
    - Create a **Bar Chart** to display Sales by Category.
    - Create a **Line Chart** to display the Monthly Sales Trend.
    - Create a **Pie Chart** to display Sales proportion by Region.
    - Create suitable tables or KPI views for important sales information.
5. Create calculated fields where required for deriving additional values from the available data.
6. Apply **Filters** to allow users to view data based on Region, Date, Category, or other relevant fields.
7. Create a new dashboard using **Dashboard → New Dashboard**.
8. Drag the required worksheets into the dashboard and arrange them in a clear and organized layout.
9. Add filters and enable interactions so that selecting a value in one visualization can update the other visualizations.

10. Add suitable titles, labels, formatting, and legends to improve the readability of the dashboard.
11. Create a **Story** by adding the required worksheets or dashboards as story points.
12. Save the Tableau workbook for future use.

OUTPUT:



RESULT:

Thus, the required data was successfully connected, analyzed, visualized, and presented through interactive charts, dashboards, and a story using Tableau.